

explain strategy design pattern in detail in java

Strategy Design Pattern

The **Strategy Design Pattern** is a **behavioral design pattern** that lets you define **multiple algorithms/behaviors**, put each one in a separate class, and make them **interchangeable at runtime**.

Simple definition

Strategy = Encapsulate different ways of doing something behind a common interface, and let the client choose which way to use.

1. The problem Strategy solves

Suppose you have a payment system.

You support:

- Credit Card
- UPI
- PayPal

A simple implementation might look like this:

```
class PaymentService {  
  
    public void pay(String type, double amount) {  
  
        if (type.equals("CREDIT_CARD")) {  
            // credit card payment logic  
        }  
        else if (type.equals("UPI")) {  
            // UPI payment logic  
        }  
        else if (type.equals("PAYPAL")) {  
            // PayPal payment logic  
        }  
    }  
}
```

This works initially.

But imagine you add:

- Debit Card
- Net Banking
- Apple Pay
- Google Pay
- Wallet

The class becomes:

```
PaymentService
|
+-- if CREDIT_CARD
+-- if UPI
+-- if PAYPAL
+-- if DEBIT_CARD
+-- if NET_BANKING
+-- if APPLE_PAY
+-- if GOOGLE_PAY
+-- ...
```



This creates several problems:

Problem 1 — Large conditional logic

The class becomes difficult to maintain.

Problem 2 — Violates Open/Closed Principle

Every new payment method requires modifying `PaymentService`.

Problem 3 — Testing becomes harder

You have to test a large class containing many unrelated algorithms.

Problem 4 — Tight coupling

`PaymentService` knows the implementation details of every payment method.

2. Strategy pattern solution

Instead of putting all algorithms inside one class:

```
PaymentService
|
```



```
+---- CreditCardPayment
|
+---- UpiPayment
|
+---- PayPalPayment
```

We define a common interface:

```
interface PaymentStrategy {

    void pay(double amount);
}
```

Then each payment algorithm becomes a separate strategy.

```
class CreditCardPayment implements PaymentStrategy {

    @Override
    public void pay(double amount) {
        System.out.println("Paying ₹" + amount + " using Credit Card");
    }
}
```

```
class UpiPayment implements PaymentStrategy {

    @Override
    public void pay(double amount) {
        System.out.println("Paying ₹" + amount + " using UPI");
    }
}
```

```
class PayPalPayment implements PaymentStrategy {

    @Override
    public void pay(double amount) {
        System.out.println("Paying ₹" + amount + " using PayPal");
    }
}
```

Now the main service doesn't need to know how each payment works.

3. Context

The class that **uses the strategy** is called the **Context**.

```
class PaymentService {  
  
    private PaymentStrategy paymentStrategy;  
  
    public PaymentService(PaymentStrategy paymentStrategy) {  
        this.paymentStrategy = paymentStrategy;  
    }  
  
    public void makePayment(double amount) {  
        paymentStrategy.pay(amount);  
    }  
}
```

Now:

```
public class Main {  
  
    public static void main(String[] args) {  
  
        PaymentStrategy strategy =  
            new UpiPayment();  
  
        PaymentService service =  
            new PaymentService(strategy);  
  
        service.makePayment(1000);  
    }  
}
```

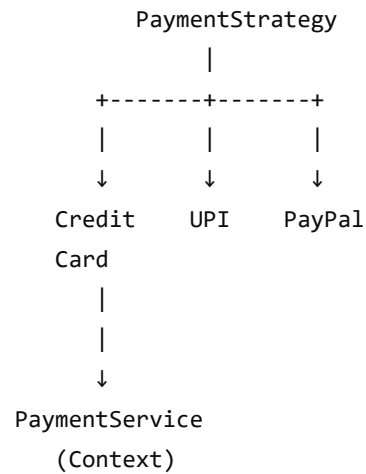
Output:

Paying ₹1000.0 using UPI



4. What is happening here?

The important relationship is:



PaymentService doesn't care whether the payment is:

UPI
Credit Card
PayPal

It only knows:

</> Java

The actual implementation is decided by the object passed to it.

5. Runtime strategy selection

This is one of the biggest advantages of Strategy.

You can change the strategy at runtime.

For example:

```

PaymentService service =
    new PaymentService(new UpiPayment());
  
```

</> Java

Later:

```
service =  
    new PaymentService(new CreditCardPayment());
```

</> Java



Or:

```
service =  
    new PaymentService(new PayPalPayment());
```

</> Java



The `PaymentService` code doesn't change.

Only the strategy changes.

6. Complete example

```
interface PaymentStrategy {  
  
    void pay(double amount);  
}
```

</> Java



UPI strategy

```
class UpiPayment implements PaymentStrategy {  
  
    @Override  
    public void pay(double amount) {  
        System.out.println(  
            "Processing UPI payment: ₹" + amount  
        );  
    }  
}
```

Credit Card strategy

```
class CreditCardPayment implements PaymentStrategy {  
  
    @Override  
    public void pay(double amount) {  
        System.out.println(  
            "Processing Credit Card payment: ₹" + amount  
        );  
    }  
}
```

</> java



PayPal strategy

```
class PayPalPayment implements PaymentStrategy {  
  
    @Override  
    public void pay(double amount) {  
        System.out.println(  
            "Processing PayPal payment: ₹" + amount  
        );  
    }  
}
```

Context

```
class PaymentService {  
  
    private final PaymentStrategy strategy;  
  
    public PaymentService(PaymentStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void makePayment(double amount) {  
        strategy.pay(amount);  
    }  
}
```

Client

```
public class Main {  
  
    public static void main(String[] args) {
```

```
PaymentStrategy strategy =  
    new UpiPayment();  
  
PaymentService service =  
    new PaymentService(strategy);  
  
service.makePayment(500);  
}  
}
```

Output:

Processing UPI payment: ₹500.0



7. Strategy Pattern structure

There are normally three important components.

1. Strategy

Common interface.

```
interface PaymentStrategy {  
    void pay(double amount);  
}
```

</> Java



2. Concrete Strategies

Different implementations.

```
class UpiPayment implements PaymentStrategy {  
    ...  
}  
  
class CreditCardPayment implements PaymentStrategy {  
    ...  
}  
  
class PayPalPayment implements PaymentStrategy {  
    ...  
}
```

3. Context

Uses the strategy.

```
class PaymentService {

    private PaymentStrategy strategy;

    ...

}
```

So remember:

Strategy
↓
defines common behavior

Concrete Strategy
↓
implements different algorithms

Context
↓
uses one strategy



8. Why not inheritance?

You could try:

```
PaymentService
|
+---- UpiPaymentService
+---- CreditCardPaymentService
+---- PayPalPaymentService
```



But this creates inheritance-based designs where the behavior is tied to the class hierarchy.

Strategy uses **composition** instead:

```
PaymentService
|
| has-a
|
↓
PaymentStrategy
|
+---- UPI
```



```
+--- Credit Card  
+--- PayPal
```

This is the important principle:

Favor composition over inheritance.

The Context has a strategy rather than being a strategy.

9. Strategy with setter

The strategy doesn't necessarily have to be supplied through the constructor.

You can also change it later.

```
class PaymentService {  
  
    private PaymentStrategy strategy;  
  
    public void setStrategy(PaymentStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public void makePayment(double amount) {  
        strategy.pay(amount);  
    }  
}
```

Then:

```
PaymentService service = new PaymentService();  
  
service.setStrategy(new UpiPayment());  
service.makePayment(1000);  
  
service.setStrategy(new CreditCardPayment());  
service.makePayment(2000);
```

Here the strategy changes at runtime.

10. Real-world example — Sorting

Suppose an application supports different sorting algorithms:

Bubble Sort
Quick Sort
Merge Sort



Create:

```
interface SortStrategy {  
  
    void sort(int[] numbers);  
}
```



Implement:

```
class QuickSort implements SortStrategy {  
  
    @Override  
    public void sort(int[] numbers) {  
        System.out.println("Using Quick Sort");  
    }  
}
```

```
class MergeSort implements SortStrategy {  
  
    @Override  
    public void sort(int[] numbers) {  
        System.out.println("Using Merge Sort");  
    }  
}
```

Context:

```
class SortService {  
  
    private final SortStrategy strategy;  
  
    public SortService(SortStrategy strategy) {  
        this.strategy = strategy;  
    }  
}
```

```
public void sort(int[] numbers) {  
    strategy.sort(numbers);  
}  
}
```

Usage:

```
SortService service =  
    new SortService(new QuickSort());
```

```
.....
```

```
</> Java
```



You can replace it with:

```
SortService service =
```

```
</> Java
```



without changing SortService .

11. Strategy vs if-else

Without Strategy:

```
if (type.equals("UPI")) {  
    upiPayment();  
} else if (type.equals("CARD")) {  
    cardPayment();  
} else if (type.equals("PAYPAL")) {  
    paypalPayment();  
}
```

With Strategy:

```
</> Java
```



The difference is:

Without Strategy



Service

```

├─ UPI logic
├─ Card logic
├─ PayPal logic
└─ More logic
  
```

With Strategy

Service

```

└─ PaymentStrategy
    ├── UPI
    ├── Card
    └─ PayPal
  
```

The second design is much easier to extend.

12. Strategy and Open/Closed Principle

Suppose tomorrow you add:

AmazonPay



Without Strategy:

```

if (type.equals("AMAZON_PAY")) {
    // new logic
}
  
```

</> Java



You must modify existing code.

With Strategy:

```

class AmazonPayPayment implements PaymentStrategy {

    @Override
    public void pay(double amount) {
        // Amazon Pay Logic
    }
}
  
```

That's it.

The existing `PaymentService` doesn't need modification.

This is why Strategy works well with the **Open/Closed Principle**:

Open for extension, closed for modification.

13. Strategy in Spring Boot

This is particularly important for a Java/Spring developer.

Spring's dependency injection makes Strategy very natural.

For example:

```
public interface PaymentStrategy {  
  
    void pay(double amount);  
  
    ,  
  
    // ...  
}
```

```
@Component  
public class UpiPaymentStrategy  
    implements PaymentStrategy {  
  
    @Override  
    public void pay(double amount) {  
        System.out.println("UPI payment");  
    }  
}
```

```
@Component  
public class CardPaymentStrategy  
    implements PaymentStrategy {  
  
    @Override  
    public void pay(double amount) {  
        System.out.println("Card payment");  
    }  
}
```

</> Java



You can inject multiple implementations:

```
@Service
public class PaymentService {

    private final List<PaymentStrategy> strategies;

    public PaymentService(
        List<PaymentStrategy> strategies) {

        this.strategies = strategies;
    }
}
```

Spring creates:

```
PaymentService
    |
    ↓
List<PaymentStrategy>
    |
    +--- UpiPaymentStrategy
    |
    +--- CardPaymentStrategy
    |
    +--- PayPalPaymentStrategy
```



Then you can select the appropriate strategy.

A common production approach is to give each strategy a key:

```
public interface PaymentStrategy {

    String getType();

    void pay(double amount);
}
```

Implementation:

```
@Component
public class UpiPaymentStrategy
    implements PaymentStrategy {

    @Override
    public String getType() {
        return "UPI";
    }

    @Override
    public void pay(double amount) {
        System.out.println("UPI payment");
    }
}
```

Then create a map:

```
@Service
public class PaymentService {

    private final Map<String, PaymentStrategy> strategies;

    public PaymentService(List<PaymentStrategy> strategyList) {

        this.strategies = strategyList.stream()
            .collect(Collectors.toMap(
                PaymentStrategy::getType,
                Function.identity()
            ));
    }

    public void pay(
        String type,
        double amount) {

        PaymentStrategy strategy =
            strategies.get(type);

        if (strategy == null) {
            throw new IllegalArgumentException(
                "Unsupported payment type: " + type
            );
        }

        strategy.pay(amount);
    }
}
```

Now:


```
</> Java
```



or:

```
</> Java
```



This is a very common real-world use of Strategy in Spring Boot.

14. Where you commonly see Strategy in real applications

For a Java/Spring Boot developer, look for situations like:

Payment

```
UPI  
Credit Card  
Debit Card  
Net Banking
```



Pricing

```
Regular customer pricing  
Premium customer pricing  
VIP pricing  
Festival pricing
```



Shipping

```
Standard shipping  
Express shipping  
International shipping  
Same-day shipping
```



Notification

```
Email  
SMS  
Push notification  
WhatsApp
```



Authentication

```
Password authentication
OTP authentication
OAuth authentication
API-key authentication
```



File processing

```
CSV
JSON
XML
Excel
```



Insurance

For example:

```
PremiumCalculationStrategy
|
+--- LifeInsurancePremium
+--- HealthInsurancePremium
+--- TermInsurancePremium
```



Each algorithm can be independently implemented.

15. When should you use Strategy?

Use Strategy when:

1. You have multiple algorithms

```
Algorithm A
Algorithm B
Algorithm C
```



and they perform the same conceptual operation.

2. You have a large if/else or switch

Especially when the branches contain substantial business logic.

3. Behavior changes at runtime

For example:

User selects payment method
↓
Choose corresponding strategy



4. You expect new behaviors

If new implementations will probably be added frequently, Strategy is a good candidate.

5. You want independent testing

Each strategy can be unit-tested separately.

16. When NOT to use Strategy

Don't create Strategy classes for every small `if`.

Bad example:

```
if (age > 18) {  
    ...
```

</> Java



Creating:

```
AdultStrategy  
MinorStrategy
```



would usually be unnecessary.

Strategy is useful when the alternatives represent **meaningful, replaceable algorithms or business behaviors**.

17. Strategy vs Factory

This is a common interview question.

Factory

Factory answers:

Which object should I create?

Example:

```
PaymentStrategy strategy =
```

```
</> Java
```



Strategy

Strategy answers:

Which algorithm/behavior should I execute?

Example:

```
</> Java
```



They are often used **together**.

```
Client
|
↓
Factory
|
↓
creates Strategy
|
↓
Context
|
↓
executes Strategy
```



18. Strategy vs State

Another common interview question.

They look similar because both often use:

```
interface
↓
```

```
</> Java
```



But their intent is different.

Strategy

The client/application chooses the algorithm.

Choose payment method
↓
UPI Strategy



State

The object's **internal state** determines its behavior.

For example:

Order
|
+-- CREATED
|
+-- PAID
|
+-- SHIPPED
|
+-- DELIVERED



The key difference:

Strategy → interchangeable algorithms



State → behavior changes because object's state changes

19. Strategy vs Template Method

Strategy

Uses **composition**.

Context
|
↓
Strategy



Different algorithms are separate objects.

Template Method

Uses **inheritance**.

```

AbstractClass
|
+--- Implementation A
|
+--- Implementation B

```



Template Method defines the overall algorithm structure and allows subclasses to customize certain steps.

So:

```

Strategy      → composition
Template Method → inheritance

```



20. Key interview points

For a 7–8 year Java developer, remember these points:

1. Strategy is a behavioral design pattern.
2. It encapsulates interchangeable algorithms/behaviors.
3. It eliminates or reduces large conditional logic.
4. It uses **composition over inheritance**.
5. Strategy implementations usually share a common interface.
6. The Context delegates behavior to the selected strategy.
7. Strategy can be selected or changed at runtime.
8. It supports the **Open/Closed Principle**.
9. In Spring Boot, multiple `@Component` implementations can be injected as a `List` or `Map`.
10. Factory chooses/creates the strategy; Strategy performs the behavior.

The one-line mental model

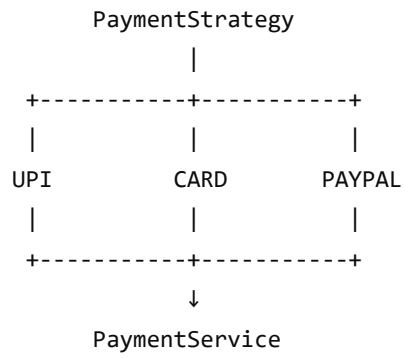
```

Many ways to perform the SAME operation
↓
Common interface
↓
Separate implementations
↓
Context chooses one

```



For example:



That is Strategy Design Pattern.



ChatGPT can make mistakes. Check important info.